

Functional Requirements

User Authentication & Access Control

- **User Registration and Login:** The system shall allow new users to register by providing a unique email address and password. Registered users (Admins or Team Members) can log in to access the platform. Passwords must be securely stored (e.g. hashed with bcrypt) and users must be able to reset forgotten passwords via email verification. User registration/login is a core feature: “Users should be able to register and log in...”.
- **Role-Based Access Control (RBAC):** The system defines two roles **Administrator** and **Team Member** with distinct permissions. Administrators can access all management features, while Team Members have limited views (e.g. only see their own tasks and data). Permissions are assigned to roles, not individuals, ensuring users “only access what they need for their work”. This RBAC ensures, for example, only Admins can create or assign tasks, while Team Members can update task status and view personal analytics.

Task Management

- **Task Creation & Editing:** Administrators can create, edit, and delete tasks, specifying details such as title, description, due date, priority, and assigned team member.
- **Task Assignment:** Admins can assign tasks to specific team members. The system also provides AI-driven assignment suggestions: when creating a task, the system recommends the “most suitable team member” based on current workload, past performance, and mood patterns.
- **Task Tracking:** Team Members can view their assigned tasks in a dashboard, update status (e.g. “To Do”, “In Progress”, “Done”), and mark tasks complete. The system tracks task progress and completion history. Automated reminders or notifications are sent for upcoming deadlines.
- **Performance Logging:** Task completion times and outcomes are recorded. Managers (Admins) can review team workload and productivity metrics (e.g. tasks completed per period) to make informed assignment decisions.

Emotion Tracking

- **Daily Mood Logging:** Each user (Team Member) can log their daily mood using both qualitative and quantitative inputs. For example, an **emoji slider** (happy→sad) and a self-rating scale allow quick mood entries. Optionally, users can add a short text or journal entry describing their feelings or context.
- **Sentiment Analysis:** Textual mood entries are analyzed using a pre-trained NLP model (e.g. a BERT transformer from Hugging Face). The system extracts sentiment or emotional tone from free-text inputs to categorize entries (positive, neutral, negative) and detect trends.
- **Behavioral Pattern Detection:** The platform stores all mood entries over time to compute individual trends. It identifies patterns (e.g. consistently low mood on certain days) and correlates them with task data. This analysis is used later for adaptive features and reports.

- **Mood Visualization:** Users and managers see visual charts of mood history (e.g. weekly mood averages, emotion distribution). The dashboard highlights significant changes in emotional state and can alert if a user's mood drops sharply.

Gamification

- **Points and Rewards:** The system awards points for desired actions, such as completing tasks, consistently logging mood, or achieving performance targets. Points accumulate over time.
- **Levels and Badges:** As users earn points, they progress through levels. Milestones (e.g. "First 100 points" or "10 tasks completed") trigger badges or achievements. These visual rewards (badges) are displayed on the user's profile.
- **Adaptive Difficulty:** Gamification elements adjust based on user emotion trends. For example, if a user reports low mood over several days, the system might reduce point thresholds or assign fewer simultaneous tasks to avoid overload. Conversely, high-mood periods might unlock extra challenges. In short, points/badges/levels "dynamically adjust based on emotional state" to keep engagement optimal.
- **Leaderboards and Social Features:** Optionally, the system can include a leaderboard (visible to Admins or teams) to foster friendly competition. It ranks users by points or badges earned, encouraging positive gamified behavior (configurable for privacy).

Insights & Feedback

- **AI-Generated Coaching:** An integrated LLM (e.g. GPT or Gemini) provides personalized feedback to each user. Based on the user's mood history and performance data, the system generates motivational messages and suggestions. For instance, if a user feels demotivated, the AI might offer encouragement or tips ("Take a short break and try these steps..."). This aligns with the proposal's goal that "an integrated LLM ... will then provide personalized feedback, encouragement, and productivity suggestions tailored to each user's emotional trends".
- **Task Recommendations:** The LLM also suggests tasks or re-prioritizations. For example, if a user is nearing burnout, the AI might advise rescheduling non-urgent tasks or delegating work. These recommendations help managers and team members adjust work proactively.
- **Analytics Dashboard:** The system presents visual dashboards showing key metrics: task completion rates, average mood over time, correlation between mood and productivity, etc. Admins can filter by user or team to spot trends. All analytics aim to improve well-being and performance.
- **Reporting:** Administrators can generate reports (periodic or on-demand) that summarize team health: workload balance, collective mood trends, and areas needing attention (e.g. consistently overwhelmed team members). These reports combine quantitative data (tasks, time spent) with qualitative insights (mood logs) for a holistic view.

Calendar & Email Integration (Planned)

- **Calendar Sync:** The system will offer integration with calendar services (e.g. Google Calendar, Outlook) via APIs. Tasks with dates can automatically appear as events/Reminders in the user's personal calendar.

- **Email Notifications:** Users receive email alerts for critical events: new task assignments, upcoming deadlines, missed daily mood logs, or motivational feedback. Email can also provide one-click links back into the app.
- **Contact Integration:** (Future) The CRM may import contact lists or task assignments from linked email accounts, streamlining setup for teams already using email workflows.

Non-Functional Requirements

Performance

- **Responsiveness:** The web UI should load quickly (e.g. main pages <2 seconds) and react promptly to user actions. Page transitions and data fetches must not feel laggy. The LLM and sentiment analysis calls (which may run server-side) should produce feedback within a few seconds to maintain user engagement.
- **Concurrency:** The system must support at least hundreds (ideally thousands) of concurrent users without significant slowdown. For example, it should comfortably handle $\geq 1,000$ simultaneous users (typical for an enterprise prototype) while keeping page loads under ~ 1 second. Backend APIs (Node.js/MongoDB) should use efficient queries and caching to meet these targets.
- **Real-Time Updates:** Task assignments, points, and mood logs should propagate in near-real-time. For instance, when an Admin assigns a task, the assigned user's task list updates immediately (via WebSocket or polling). Gamification updates (points/badges) appear without requiring full page refreshes.

Scalability

- **Horizontal Scalability:** Architect the system on the MERN stack, which is inherently scalable. The application must be deployable in a cloud environment (e.g. MongoDB Atlas, Node containers) so that additional instances can be added as load grows. The database (MongoDB) should be configured for sharding or clustering to handle increasing data (many users, tasks, logs).
- **Modular Design:** Separate concerns so components can scale independently. For example, run the Node/Express API, the AI/ML backend (Python microservice), and the front-end server as distinct modules. This allows adding more resources to the AI service during heavy analysis or to the front-end servers as traffic grows.
- **Load Balancing:** The system should use load balancers (e.g. Nginx, Kubernetes) to distribute incoming requests across server instances. Automatic scaling policies (cloud autoscaling) can be set up to add capacity during peak usage.

Reliability & Availability

- **Uptime:** Aim for high availability (e.g. $\geq 99.9\%$ uptime). Deploy on resilient infrastructure with redundancy. Automated monitoring should alert admins of any downtime or performance degradation.
- **Fault Tolerance:** The system should handle failures gracefully. For instance, if the LLM service is temporarily unavailable, the UI should still function (perhaps queuing feedback requests). Uncaught exceptions must be logged and should not crash the server.

- **Data Backup & Recovery:** Regular backups of the database must be performed (e.g. daily snapshots). In case of data loss or corruption, the system can restore from backups with minimal data loss (RPO/RTO targets to be defined). Audit logs (for tasks changes, user logins) should be maintained to trace any issues.
- **Error Handling:** The UI should provide helpful error messages (e.g. form validation errors) and the backend should validate all inputs. Failures (e.g. network errors) should trigger retries or user-friendly prompts.
- **Testing & Quality:** Automated tests (unit/integration) should cover core modules (auth, task mgmt, mood logging, AI calls). Continuous Integration (CI) pipelines will run these tests on each commit to catch regressions early.

Security & Privacy

- **Data Protection:** All sensitive data (user credentials, personal mood logs) must be encrypted. Passwords are hashed (bcrypt) and never stored in plaintext. Data in transit is protected via HTTPS (TLS). Sensitive fields in the database should use encryption or access controls.
- **Authentication Security:** Implement secure session management (e.g. JWT with proper expiration, secure cookies). Protect against common web attacks (XSS, CSRF, SQL/NoSQL injection) by using established libraries and frameworks (Express's helmet, input sanitizers, etc.).
- **Access Control:** RBAC ensures users can only access permitted resources. For instance, only Admin endpoints accept task creation requests. API endpoints check the user's role token before performing actions. Regular audits of role permissions help avoid "permission creep."
- **Privacy Compliance:** The platform should allow users to delete or anonymize their personal data if requested (GDPR compliance). Terms of use and privacy policies must be clear about how mood/emotion data is used.
- **Audit Logging:** Record critical actions (logins, data changes) with timestamps and user IDs. This supports forensic analysis if a breach or misuse is suspected.
- **Dependency Management:** Keep all libraries (NPM packages, Python libraries) updated. Use security scanning tools to detect vulnerabilities in dependencies.

Usability & Accessibility

- **Intuitive UI:** The user interface (built with React/Tailwind CSS) should be clean and user-friendly. Navigation menus and buttons must be clearly labeled. Use consistent design patterns so users learn the workflow quickly. Onboarding tutorials or tooltips can help first-time users.
- **Responsive Design:** The web app must work on various screen sizes (desktops, tablets, smartphones). Layouts should adapt so important dashboards and forms are legible and usable on smaller screens.
- **Accessibility:** Follow WCAG 2.1 guidelines. For example, ensure sufficient color contrast, support keyboard navigation, and provide alt-text for images. Emoji sliders and charts should have text alternatives. This makes the tool usable by people with disabilities.
- **Performance Perception:** Even if some operations take time (e.g. running sentiment analysis), the UI should give feedback (loading spinners, progress bars) so users know the system is working.

- **Internationalization (Future):** Although initially English-only, design text and layouts so that adding other languages later (i18n) is straightforward.

Maintainability & Extensibility

- **Modular Architecture:** Code should follow the MERN conventions with clear separation of front-end (React components), backend API (Express routes/controllers), and database layers. This makes it easier to update one part without affecting others.
- **Documentation:** Maintain up-to-date documentation: inline code comments, a README for setup instructions, and API documentation (e.g. OpenAPI/Swagger for REST endpoints). Clearly document algorithms for emotion analysis and gamification rules.
- **Configuration Management:** Use environment variables (e.g. via dotenv) for configurable settings (database URLs, API keys). This avoids hard-coding and simplifies deployment to new environments.
- **Automated Testing:** Include unit and integration tests for key functions (e.g. authentication flow, task assignment logic, sentiment analysis output). A high test coverage ensures confidence in future changes.
- **CI/CD Pipeline:** Use version control (GitHub) with a CI pipeline that runs tests and lints code on every commit. This helps catch bugs early and enforces code quality standards.
- **Extensibility:** The system should allow adding new features with minimal refactoring. For example, the gamification module could later support new reward types, or new emotion analysis models could be plugged in. Design interfaces (e.g. an abstract analytics service) to accommodate such growth.

Performance, Reliability, and Other Quality Attributes

- **System Availability:** Target “five nines” (99.999%) or at least 99.9% uptime once in production. Use cloud services (like MongoDB Atlas, Azure/GCP) that offer SLAs.
- **Scalable Infrastructure:** As mentioned, use containerization (Docker) and orchestration (Kubernetes) so the system can scale horizontally. This addresses future enterprise deployment needs.
- **Backup & Monitoring:** Automated backups of data and logs. Real-time monitoring (with tools like Prometheus/New Relic) for CPU, memory, and custom metrics (e.g. queue lengths) to alert on anomalies.
- **Performance Benchmarks:** Before release, conduct load testing (e.g. using JMeter) to validate that the app meets performance NFRs (response times, concurrent users). For example, ensure the web UI renders task lists in <500ms and that 1,000 concurrent users can log moods or view dashboards smoothly.
- **Maintainability Metrics:** Use code linting and static analysis (ESLint, SonarQube) to track code quality. Aim for low technical debt by refactoring code where needed.

By addressing these **functional requirements** (organized by feature modules) and **non-functional requirements** (covering performance, scalability, reliability, usability, maintainability, security, etc.), the Emotion-Aware Gamified CRM will meet user needs for task and emotion management today, while being robust and adaptable for future enterprise-scale deployment

